

1)

```
temperaturas = []
```

```
leituras = [820, 835, 848, 791, 812, 856]
```

```
for temp in leituras:
```

```
    temperaturas.append(temp)
```

```
print(f"Leituras: {temperaturas}")
```

```
print(f"Máximo: {max(temperaturas)} °C")
```

```
print(f"Mínimo: {min(temperaturas)} °C")
```

2)

```
amostras = [11.9, 12.1, 12.0, 13.5, 11.7, 12.3, 10.5, 12.0]
```

```
leituras = []
```

```
for v in amostras:
```

```
    leituras.append(v)
```

```
media = sum(leituras) / len(leituras)
```

```
print(f"Média: {media:.2f} V")
```

```
for v in leituras:
```

```
    if v < 10.8 or v > 13.2:
```

```
        print(f"ALERTA: {v} V está fora da faixa!")
```

3)

```
arquivosbytes = (512, 1024, 1500, 1518, 256, 1500, 2048, 64, 1500, 1024)
```

```
buffer = []
```

```
for x in arquivosbytes:
```

```
    buffer.append(x)
```

```
jumbo = [x for x in buffer if x >= 1500]
```

```
porcentagem = (len(jumbo) / len(buffer)) * 100
```

```
print(f" pacote jumbo{len(jumbo)} ({porcentagem:.1f}%)")
```

```
print(f" pacotes totais {len(buffer)}")
```

```
print(f" total bytes {sum(buffer)}")
```

4)

```
vibracao = [0.12, 0.34, -999.0, 0.28, 0.91, -999.0, 1.42, 0.67]
```

```
while -999.0 in vibracao:
```

```
vibracao.remove(-999.0)
```

```
print("Leituras válidas:", vibracao)  
print("Quantidade de leituras válidas:", len(vibracao))
```

5)

```
cargas = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]  
antes = sum(cargas)  
cargas.remove(40.0)  
cargas.remove(8.0)  
depois = sum(cargas)
```

```
print("Cargas ainda ativas:", cargas)  
print("Potência de antes:", antes, "kW")  
print("Potência de agora:", depois, "kW")  
print("Redução da potencia de:", antes - depois, "kW")
```

6)

```
canais = [850, 900, 950, 1800, 2100, 2600]  
interferencias = {850:-65, 900:-80, 950:-72, 1800:-55, 2100:-90, 2600:-68}  
remover = []  
for c in canais:  
    if interferencias[c] >= -72:  
        print(f"Canal {c} MHz desativado (interferência: {interferencias[c]} dBm)")  
        remover.append(c)  
for r in remover:  
    canais.remove(r)  
print("Canais ativos:", canais)
```

7)

```
params = [120.0, 150.0, 95.0, 200.0, 110.0]  
backup = params.copy()  
params[0] = 130.0  
print("Original (modificada):", params)  
print("Backup (preservado):", backup)
```

8)

```
nominal = [47.0, 100.0, 220.0, 33.0, 68.0]  
simulacao = nominal.copy()  
s = simulacao.index(100.0)  
simulacao[s] = 0.0  
req_nominal = sum(nominal)  
req_simulacao = sum(simulacao)
```

```
print("Nominal:", nominal)
print("Simulação:", simulacao)
print("Req nominal:", req_nominal, "Ω")
print("Req falha:", req_simulacao, "Ω")
print("Variação:", req_nominal - req_simulacao, "Ω")
```

9)

```
antiga = ['10.0.0.0/8', '172.16.0.0/12', '192.168.1.0/24', '203.0.113.0/24', '198.51.100.0/24']
nova = antiga.copy()
nova.remove('203.0.113.0/24')
nova.extend(['10.10.0.0/16', '192.168.100.0/24'])
adicionadas = [r for r in nova if r not in antiga]
removidas = [r for r in antiga if r not in nova]
print("Rotas adicionadas:", adicionadas)
print("Rotas removidas:", removidas)
```

10)

```
sensorA = [3.2, 3.4, 3.1, 3.5]
sensorB = [4.1, 4.0, 4.3, 3.9]
sensorC = [2.8, 2.9, 3.0, 2.7]
todas = []
todas.extend(sensorA)
todas.extend(sensorB)
todas.extend(sensorC)
print("Total de leituras:", len(todas))
print("preço média:", sum(todas)/len(todas))
print("preço máxima:", max(todas))
print("preço mínima:", min(todas))
```

11)

```
media1 = [48.2, 51.0, 49.7, 47.5, 52.3]
media2 = [50.1, 46.8, 53.0, 49.0, 55.2]
dados = media1.copy()
dados.extend(media2)
dados.sort()
media = sum(dados)/len(dados)
tolerancia = media * 0.15
```

```
print("medições ordenadas:", dados)
print(f"Média: {media:.2f} Ω | Tolerância: + ou -{tolerancia:.2f}")
for v in dados:
    if abs(v - media) > tolerancia:
        print("nenhum valor fora da faixa neste conjunto:", v)
```

12)

```
norte = [-85, -90, -105, -78, -92, -110, -88]
sul = [-95, -88, -102, -79, -85, -91, -97]
leste = [-80, -75, -108, -93, -86, -103, -82]
dados = []
dados.extend(norte)
dados.extend(sul)
dados.extend(leste)
total = len(dados)
cobertura = sum(1 for x in dados if x >= -100)
pior = min(dados)
indice = dados.index(pior)
media = sum(dados)/total

print("Total de amostra:", total)
print("Em cobertura >= -100:", cobertura)
print("Taxa de cobertura:", cobertura/total*100, "%")
print("Pior sinal:", pior, "[índice]= ", indice)
print("Média:", media)
```

13)

```
medidas = [50.02, 49.97, 50.06, 49.94, 50.00, 50.08, 49.99, 50.01, 49.92, 50.04]
diametronominal = 50.00
tolerancia = 0.05
aprovadas = []
reprovadas = []
for m in medidas:
    if abs(m - diametronominal) <= tolerancia:
        aprovadas.append(m)
    else:
        reprovadas.append(m)
print("Aprovadas:", aprovadas)
print("Reprovadas:", reprovadas)
print("Índice de aprovação:", len(aprovadas)/len(medidas)*100, "%")
```

14)

```
cenarioatual = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
novas = [55.0, 18.0, 75.0]
cenariofuturo = cenarioatual.copy()
cenariofuturo.extend(novas)

print("--- cenario atual ---")
```

```
print("total=", sum(cenarioatual), "média=", sum(cenarioatual)/len(cenarioatual), "maior=",  
max(cenarioatual))  
print("--- cenario futuro ---")  
print("total=", sum(cenariofuturo), "média=", sum(cenariofuturo)/len(cenariofuturo), "maior=",  
max(cenariofuturo))
```

15)precisei de ajuda da LLM Gemini para resolver
import math

```
canal_I = [0.82, 0.79, 0.85, 5.20, 0.81, 0.78, 0.83]  
canal_Q = [0.41, 0.39, 0.43, 0.40, -4.80, 0.42, 0.38]  
sinal = canal_I.copy()  
sinal.extend(canal_Q)  
backup = sinal.copy()  
media = sum(sinal) / len(sinal)  
variancia = sum((x - media)**2 for x in sinal) / len(sinal)  
desvio = math.sqrt(variancia)  
ruidos = []
```

```
for x in sinal:  
    if abs(x - media) > 3 * desvio:  
        ruidos.append(x)
```

```
for r in ruidos:  
    sinal.remove(r)
```

```
print("Sinal bruto (", len(backup), "amostras):", backup)  
print("Ruídos removidos:", ruidos)  
print("Sinal limpo (", len(sinal), "amostras):", sin
```

onlinegdb.com/#

Language Python 3

```
1 antiga = ['10.0.0.0/8', '172.16.0.0/12', '192.168.1.0/24', '203.0.113.0/24', '198.51.100.0/24']
2 nova = antiga.copy()
3 nova.remove('203.0.113.0/24')
4 nova.extend(['10.10.0.0/16', '192.168.100.0/24'])
5 adicionadas = [r for r in nova if r not in antiga]
6 removidas = [r for r in antiga if r not in nova]
7 print("Rotas adicionadas:", adicionadas)
8 print("Rotas removidas:", removidas)
```

close ad [x]

Economize tempo em edições complexas com recursos de IA.

Desconto especial no plano Pro.

[Compre agora](#)

Adobe Creative Cloud Pro

input stdout stderr

Compiled Successfully. memory: 11904 time: 0.04 exit code: 0

Rotas adicionadas: ['10.10.0.0/16', '192.168.100.0/24']
Rotas removidas: ['203.0.113.0/24']

onlinegdb.com/#

Language Python 3

```
1 sensorA = [3.2, 3.4, 3.1, 3.5]
2 sensorB = [4.1, 4.0, 4.3, 3.9]
3 sensorC = [2.8, 2.9, 3.0, 2.7]
4 todas = []
5 todas.extend(sensorA)
6 todas.extend(sensorB)
7 todas.extend(sensorC)
8 print("Total de leituras:", len(todas))
9 print("preção média:", sum(todas)/len(todas))
10 print("preção máxima:", max(todas))
11 print("preção mínima:", min(todas))
```

close ad [x]

Economize tempo em edições complexas com recursos de IA.

Desconto especial no plano Pro.

[Compre agora](#)

Adobe Creative Cloud Pro

input stdout stderr

Compiled Successfully. memory: 11520 time: 0.05 exit code: 0

Total de leituras: 12
preção média: 3.4083333333333333
preção máxima: 4.3
preção mínima: 2.7

onlinegdb.com/#

main.py

```
1 media1 = [48.2, 51.0, 49.7, 47.5, 52.3]
2 media2 = [50.1, 46.8, 53.0, 49.0, 55.2]
3 dados = media1.copy()
4 dados.extend(media2)
5 dados.sort()
6 media = sum(dados)/len(dados)
7 tolerancia = media * 0.15
8
9 print("medições ordenadas:", dados)
10 print(f"Média: {media:.2f} | Tolerância: + ou - {tolerancia:.2f}")
11 for v in dados:
12     if abs(v - media) > tolerancia:
13         print("nenhum valor fora da faixa neste conjunto:", v)
```

Compiled Successfully. memory: 10496 time: 0.07 exit code: 0

medições ordenadas: [46.8, 47.5, 48.2, 49.0, 49.7, 50.1, 51.0, 52.3, 53.0, 55.2]
Média: 50.28 | Tolerância: + ou - 7.54

input stdout stderr

crie.conexoes.com

InstantTalks

CRIE CONEXÕES

onlinegdb.com/#

main.py

```
1 norte = [-85, -90, -105, -78, -92, -110, -88]
2 sul = [-95, -88, -102, -79, -85, -91, -97]
3 leste = [-80, -75, -108, -93, -86, -103, -82]
4 dados = []
5 dados.extend(norte)
6 dados.extend(sul)
7 dados.extend(leste)
8 total = len(dados)
9 cobertura = sum(1 for x in dados if x >= -100)
10 pior = min(dados)
11 indice = dados.index(pior)
12 media = sum(dados)/total
13
14 print("Total de amostra:", total)
15 print("Em cobertura >= -100:", cobertura)
16 print("Taxa de cobertura:", cobertura/total*100, "%")
17 print("Pior sinal:", pior, "[indice]= ", indice)
18 print("Média:", media)
```

Compiled Successfully. memory: 11648 time: 0.04 exit code: 0

Total de amostra: 21
Em cobertura >= -100: 16
Taxa de cobertura: 76.19047619047619 %
Pior sinal: -110 [indice]= 5
Média: -91.04761904761905

input stdout stderr

crie.conexoes.com

InstantTalks

CRIE CONEXÕES

<https://www.onlinegdb.com/#tab-stdout>

onlinegdb.com/#

Run Debug Stop Share Save Beautify

Language Python 3

main.py

```
1 medidas = [50.02, 49.97, 50.06, 49.94, 50.00, 50.08, 49.99, 50.01, 49.92, 50.04]
2 diametronominal = 50.00
3 tolerancia = 0.05
4 aprovadas = []
5 reprovadas = []
6 for m in medidas:
7     if abs(m - diametronominal) <= tolerancia:
8         aprovadas.append(m)
9     else:
10        reprovadas.append(m)
11 print("Aprovadas:", aprovadas)
12 print("Reprovadas:", reprovadas)
13 print("Índice de aprovação:", len(aprovadas)/len(medidas)*100, "%")
```

input stdout stderr

Compiled Successfully. memory: 11648 time: 0.09 exit code: 0

Aprovadas: [50.02, 49.97, 50.0, 49.99, 50.01, 50.04]
Reprovadas: [50.06, 49.94, 50.08, 49.92]
Índice de aprovação: 60.0 %

close ad [X]



CRIE CONEXÕES

onlinegdb.com/#

Run Debug Stop Share Save Beautify

Language Python 3

main.py

```
1 cenarioatual = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
2 novas = [55.0, 18.0, 75.0]
3 cenariofuturo = cenarioatual.copy()
4 cenariofuturo.extend(novas)
5
6 print("--- cenario atual ---")
7 print(sum(cenarioatual), sum(cenarioatual)/len(cenarioatual), max(cenarioatual))
8 print("--- cenario futuro ---")
9 print(sum(cenariofuturo), sum(cenariofuturo)/len(cenariofuturo), max(cenariofuturo))
```

input stdout stderr

Compiled Successfully. memory: 11648 time: 0.04 exit code: 0

--- cenario atual ---
128.0 21.333333333333332 40.0
--- cenario futuro ---
276.0 30.666666666666668 75.0

close ad [X]



CRIE CONEXÕES

onlinegdb.com/#

main.py

```
1 cenarioatual = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
2 novas = [55.0, 18.0, 75.0]
3 cenariofuturo = cenarioatual.copy()
4 cenariofuturo.extend(novas)
5
6 print("--- cenario atual ---")
7 print("total=", sum(cenarioatual), "média=", sum(cenarioatual)/len(cenarioatual), "maior=", max(cenarioatual))
8 print("--- cenario futuro ---")
9 print("total=", sum(cenariofuturo), "média=", sum(cenariofuturo)/len(cenariofuturo), "maior=", max(cenariofuturo))
```

input stdout stderr

Compiled Successfully. memory: 11392 time: 0.05 exit code: 0

```
--- cenario atual ---
total= 128.0 média= 21.333333333333332 maior= 40.0
--- cenario futuro ---
total= 276.0 média= 30.666666666666668 maior= 75.0
```

close ad [x]

InstantTalks



CRIE CONEXÕES

onlinegdb.com/#

main.py

```
1 import math
2 canal_I = [0.82, 0.79, 0.85, 5.20, 0.81, 0.78, 0.83]
3 canal_Q = [0.41, 0.39, 0.43, 0.40, -4.80, 0.42, 0.38]
4 sinal = canal_I.copy()
5 sinal.extend(canal_Q)
6 backup = sinal.copy()
7 media = sum(sinal) / len(sinal)
8 variancia = sum((x - media)**2 for x in sinal) / len(sinal)
9 desvio = math.sqrt(variancia)
10 ruidos = []
11 for x in sinal:
12     if abs(x - media) > 3 * desvio:
13         ruidos.append(x)
14 for r in ruidos:
15     sinal.remove(r)
16 print("Sinal bruto (", len(backup), "amostras):", backup)
17 print("Ruidos removidos:", ruidos)
18 print("Sinal limpo (", len(sinal), "amostras):", sinal)
```

input stdout stderr

Compiled Successfully. memory: 11776 time: 0.08 exit code: 0

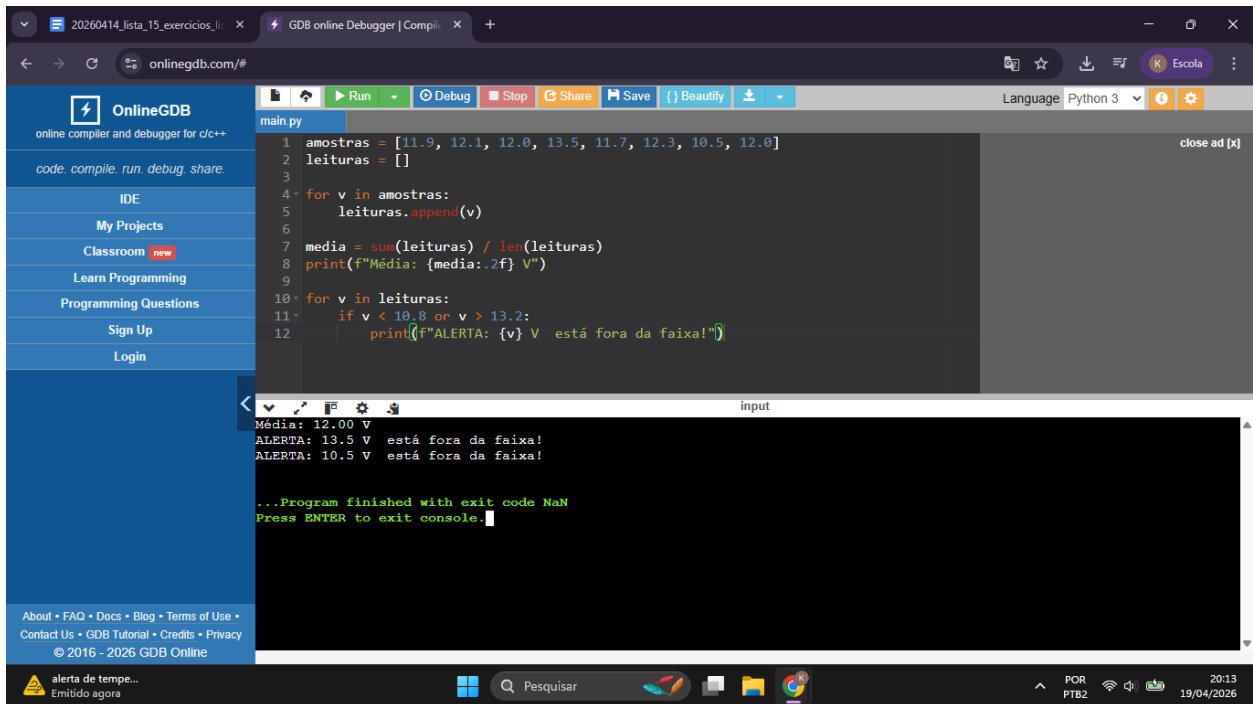
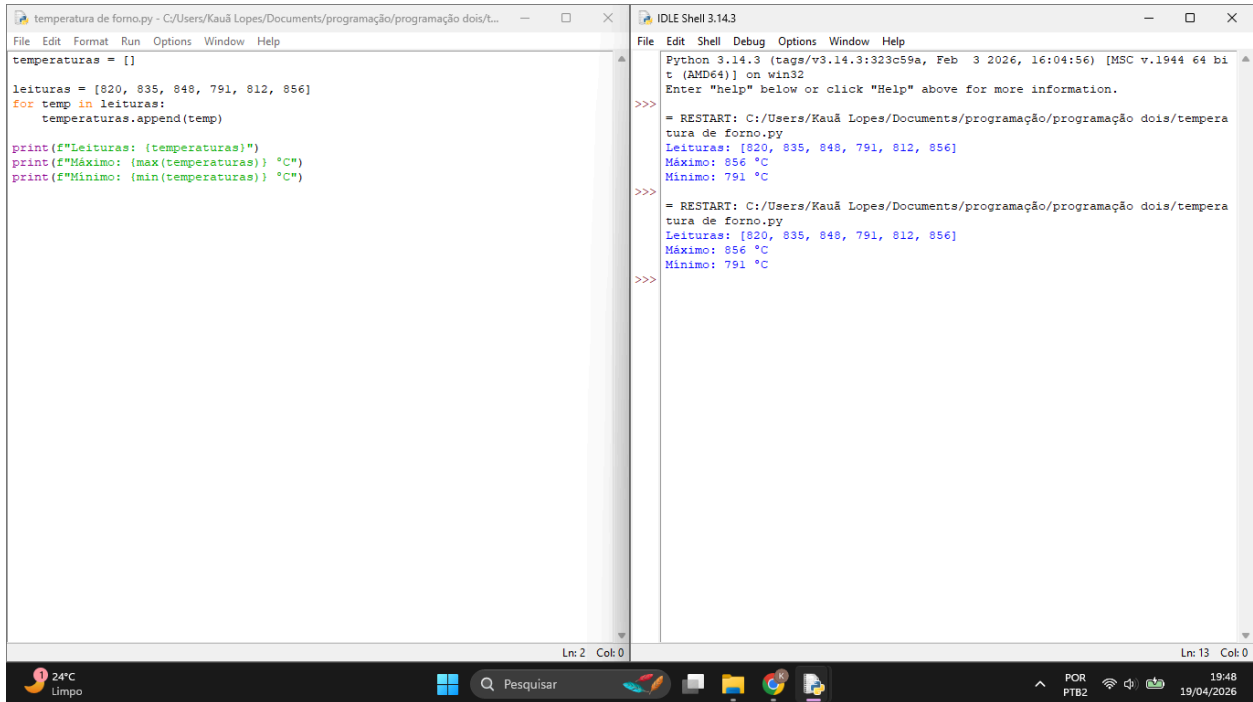
```
Sinal bruto ( 14 amostras): [0.82, 0.79, 0.85, 5.2, 0.81, 0.78, 0.83, 0.41, 0.39, 0.43, 0.4, -4.8, 0.42, 0.38]
Ruidos removidos: []
Sinal limpo ( 14 amostras): [0.82, 0.79, 0.85, 5.2, 0.81, 0.78, 0.83, 0.41, 0.39, 0.43, 0.4, -4.8, 0.42, 0.38]
```

close ad [x]

InstantTalks



CRIE CONEXÕES



20260414_lista_15_exercicios_li x GDB online Debugger | Compil x GDB online Debugger | Compil x +

onlinegdb.com/#

OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.

IDE
My Projects
Classroom **new**
Learn Programming
Programming Questions
Sign Up
Login

About • FAQ • Docs • Blog • Terms of Use •
Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

main.py

```
1 arquivosbytes = (512, 1024, 1500, 1518, 256, 1500, 2048, 64, 1500, 1024)
2 buffer = []
3
4 for x in arquivosbytes:
5     buffer.append(x)
6
7 jumbo = [x for x in buffer if x >= 1500]
8 percentagem = (len(jumbo) / len(buffer)) * 100
9
10 print(f"pacote jumbo{len(jumbo)} ({percentagem:.1f}%)")
11 print(f"pacotes totais {len(buffer)}")
12 print(f"total bytes {sum(buffer)}")
```

close ad [X]

input

```
pacote jumbo5 (50.0%)
pacotes totais 10
total bytes 10946

...Program finished with exit code 0
Press ENTER to exit console.
```

24°C
Limpó

Pesquisar

POR
PTB2

20:49
19/04/2026

Gemoetria_Analitica - Lista | x Geometria-Resumo.pdf x Resolver os exercicios de lista x 20260414_lista_15_exercicio x GDB online Debugger | Com x +

onlinegdb.com/#

OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.

IDE
My Projects
Classroom **new**
Learn Programming
Programming Questions
Sign Up
Login

About • FAQ • Docs • Blog • Terms of Use •
Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

main.py

```
1 vibracao = [0.12, 0.34, -999.0, 0.28, 0.91, -999.0, 1.42, 0.67]
2
3 while -999.0 in vibracao:
4     vibracao.remove(-999.0)
5
6 print("Leituras válidas:", vibracao)
7 print("Quantidade de leituras válidas:", len(vibracao))
```

close ad [X]

input

stdout

stderr

Compiled Successfully. memory: 11648 time: 0.04 exit code: 0

```
Leituras válidas: [0.12, 0.34, 0.28, 0.91, 1.42, 0.67]
Quantidade de leituras válidas: 6
```



OnlineGDB

online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE

My Projects

Classroom new

Learn Programming

Programming Questions

Sign Up

Login

About • FAQ • Docs • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy

© 2016 - 2026 GDB Online

main.py

```
1 cargass = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
2 antes = sum(cargass)
3 cargass.remove(40.0)
4 cargass.remove(8.0)
5 depois = sum(cargass)
6
7 print("Cargas ainda ativas:", cargass)
8 print("Potência de antes:", antes, "kW")
9 print("Potência de agora:", depois, "kW")
10 print("Redução da potencia de:", antes - depois, "kW")
```

close ad [x]

Economize tempo em edições complexas com recursos de IA.

Desconto especial no plano Pro.

Compre agora

Adobe Creative Cloud Pro

input

stdout

stderr

Compiled Successfully. memory: 11648 time: 0.09 exit code: 0

Cargas ainda ativas: [15.0, 22.5, 12.5, 30.0]
Potência de antes: 128.0 kW
Potência de agora: 80.0 kW
Redução da potencia de: 48.0 kW

OnlineGDB

online compiler and debugger for c/c++

code. compile. run. debug. share.

IDE

My Projects

Classroom new

Learn Programming

Programming Questions

Sign Up

Login

About • FAQ • Docs • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy

© 2016 - 2026 GDB Online

main.py

```
1 canais = [850, 900, 950, 1800, 2100, 2600]
2 interferencias = {850:-65, 900:-80, 950:-72, 1800:-55, 2100:-90, 2600:-68}
3 remover = []
4 for c in canais:
5     if interferencias[c] >= -72:
6         print(f"Canal {c} MHz desativado (interferência: {interferencias[c]} dBm)")
7         remover.append(c)
8 for r in remover:
9     canais.remove(r)
10 print("Canais ativos:", canais)
```

close ad [x]

Economize tempo em edições complexas com recursos de IA.

Desconto especial no plano Pro.

Compre agora

Adobe Creative Cloud Pro

input

stdout

stderr

Compiled Successfully. memory: 11136 time: 0.08 exit code: 0

Canal 850 MHz desativado (interferência: -65 dBm)
Canal 950 MHz desativado (interferência: -72 dBm)
Canal 1800 MHz desativado (interferência: -55 dBm)
Canal 2600 MHz desativado (interferência: -68 dBm)
Canais ativos: [900, 2100]

onlinegdb.com/#

OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.

IDE
My Projects
Classroom **new**
Learn Programming
Programming Questions
Sign Up
Login

About • FAQ • Docs • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

```
main.py
1 params = [120.0, 150.0, 95.0, 200.0, 110.0]
2 backup = params.copy()
3 params[0] = 130.0
4 print("Original (modificada):", params)
5 print("Backup (preservado):", backup)
```

Run Debug Stop Share Save Beautify

Language Python 3

close ad [x]

Economize tempo em edições complexas com recursos de IA.
Desconto especial no plano Pro.
Compre agora
Adobe Creative Cloud Pro

input stdout stderr

Compiled Successfully. memory: 11776 time: 0.11 exit code: 0

```
Original (modificada): [130.0, 150.0, 95.0, 200.0, 110.0]
Backup (preservado): [120.0, 150.0, 95.0, 200.0, 110.0]
```

onlinegdb.com/#

OnlineGDB
online compiler and debugger for c/c++
code. compile. run. debug. share.

IDE
My Projects
Classroom **new**
Learn Programming
Programming Questions
Sign Up
Login

About • FAQ • Docs • Blog • Terms of Use • Contact Us • GDB Tutorial • Credits • Privacy
© 2016 - 2026 GDB Online

```
main.py
1 nominal = [47.0, 100.0, 220.0, 33.0, 68.0]
2 simulacao = nominal.copy()
3 s = simulacao.index(100.0)
4 simulacao[s] = 0.0
5 req_nominal = sum(nominal)
6 req_simulacao = sum(simulacao)
7 print("Nominal:", nominal)
8 print("Simulação:", simulacao)
9 print("Req nominal:", req_nominal, "0")
10 print("Req falha:", req_simulacao, "0")
11 print("Variação:", req_nominal - req_simulacao, "0")
```

Run Debug Stop Share Save Beautify

Language Python 3

close ad [x]

Economize tempo em edições complexas com recursos de IA.
Desconto especial no plano Pro.
Compre agora
Adobe Creative Cloud Pro

input stdout stderr

Compiled Successfully. memory: 11264 time: 0.05 exit code: 0

```
Nominal: [47.0, 100.0, 220.0, 33.0, 68.0]
Simulação: [47.0, 0.0, 220.0, 33.0, 68.0]
Req nominal: 468.0 0
Req falha: 368.0 0
Variação: 100.0 0
```

al)